

Chapter 12

Retrieval Augmented Generation (RAG)

Why prompting alone breaks down, how Fine-Tuning and In-Context Learning set the stage, and how RAG combines Information Retrieval with Text Generation into a single working pipeline.

WHAT THIS CHAPTER COVERS

- ▶ Why LLMs fail on private data, recent data, and hallucinate
- ▶ Fine-Tuning as a first solution — and its drawbacks
- ▶ In-Context Learning and Few-Shot Prompting
- ▶ The definition and core idea of RAG
- ▶ The four stages: Indexing, Retrieval, Augmentation, Generation
- ▶ Indexing sub-steps: ingestion, chunking, embedding, storage
- ▶ Retriever internals: query embedding, search, ranking, fetch
- ▶ How RAG solves all three original problems, cheaply

Retrieval Augmented Generation (RAG)

RECAP — CHAPTERS 8–11

The previous four chapters built up the individual RAG components: **Document Loaders** (load data from any source), **Text Splitters** (break large text into chunks), **Vector Stores** (convert text into embeddings and store them), and **Retrievers** (perform semantic search over a vector store). This chapter assembles those pieces into the RAG architecture.

1. Why RAG Is Needed: Three Limitations of Prompting

How LLMs Store and Access Knowledge

LLMs are large transformer-based neural network architectures with a huge number of parameters (weights and biases). They are pre-trained on massive amounts of data — effectively as much text as exists on the internet. The result of this pre-training is that all of this world knowledge gets stored as numbers inside the model's weights and biases. This is why it is called *parametric knowledge*: the larger the number of parameters a model has, the more parametric knowledge it can contain, and generally the more powerful it is (e.g. a 70B parameter model can hold more knowledge than a 13B or 7B model).

As a user, the only way to access this parametric knowledge is by *prompting*: you send a query to the LLM, the model interprets the prompt, dives into its parametric knowledge, and generates a word-by-word response. This flow works well most of the time — but there are three situations where it consistently breaks down.

1. PRIVATE DATA

If you ask an LLM a question about data it never saw during pre-training — for example, a specific point inside your own company's lecture video — it simply cannot answer, because that data was never part of its training corpus. Prompting directly against private data does not work.

2. RECENT / CURRENT DATA

Every LLM has a *knowledge cut-off date* — the point up to which its training data extends. Anything that happened after that date (today's news, a newly released product) is unknown to the model. Tools like ChatGPT can answer current-affairs questions only because they have been given internet search access as an added capability — a base open-source model downloaded from a hub like Hugging Face will not be able to answer recent questions on its own.

3. HALLUCINATION

Because LLMs generate text in a fundamentally probabilistic way, there is always a chance that instead of returning factually correct information, the model confidently invents something that sounds plausible but is entirely false (for example, confidently stating that Einstein played football professionally in his early years — which never happened). This fabrication with high confidence is called *hallucination*.

IMPORTANT

These three problems — private data, recent data, and hallucination — are the motivating "why" behind both Fine-Tuning and RAG. Every subsequent technique in this chapter is judged against how well it solves these exact three problems.

Quick Revision — Why RAG Is Needed

- LLMs store all of their pre-training knowledge as numbers in their weights — this is called parametric knowledge.
- More parameters generally means more parametric knowledge and a more powerful model.
- Prompting is the only way a user can access an LLM's parametric knowledge.
- Problem 1: LLMs cannot answer questions about private data they never saw during training.
- Problem 2: Every LLM has a knowledge cut-off date and cannot answer questions about events after it.
- Problem 3: LLMs can hallucinate — generate confident but factually incorrect answers — because generation is probabilistic.

2. Fine-Tuning: The First Attempted Solution

Definition

Fine-tuning takes a pre-trained LLM and re-trains it further on a smaller, domain-specific dataset. The LLM already has broad, general-purpose knowledge from pre-training; fine-tuning adds task- or domain-specific knowledge on top of it.

Analogy

Think of an engineering graduate who has studied English, Chemistry, Physics, Computer Science, and Electronics during their degree. When this graduate joins a company, they still go through a 2–3 month company-specific training program before starting real work. The student is like the LLM; the engineering degree is like pre-training; and the company's onboarding program is like fine-tuning.

Common Fine-Tuning Techniques

Technique	Type	What It Does
Supervised Fine-Tuning (SFT)	Supervised	Trains on labeled prompt → desired-output pairs (typically 1,000 to 1,000,000+ pairs).
Continued Pre-Training	Unsupervised	Feeds unlabeled domain text (e.g. lecture transcripts) to the model in the same way pre-training happened, just on a smaller domain-specific corpus.
RLHF	Reinforcement Learning	Combines reinforcement learning with human feedback to teach the model how to behave well in real-world scenarios.
LoRA / QLoRA	Parameter-efficient	Freezes the base model weights and trains only a small set of additional weights, making fine-tuning cheaper.

How Supervised Fine-Tuning Works (4 Steps)

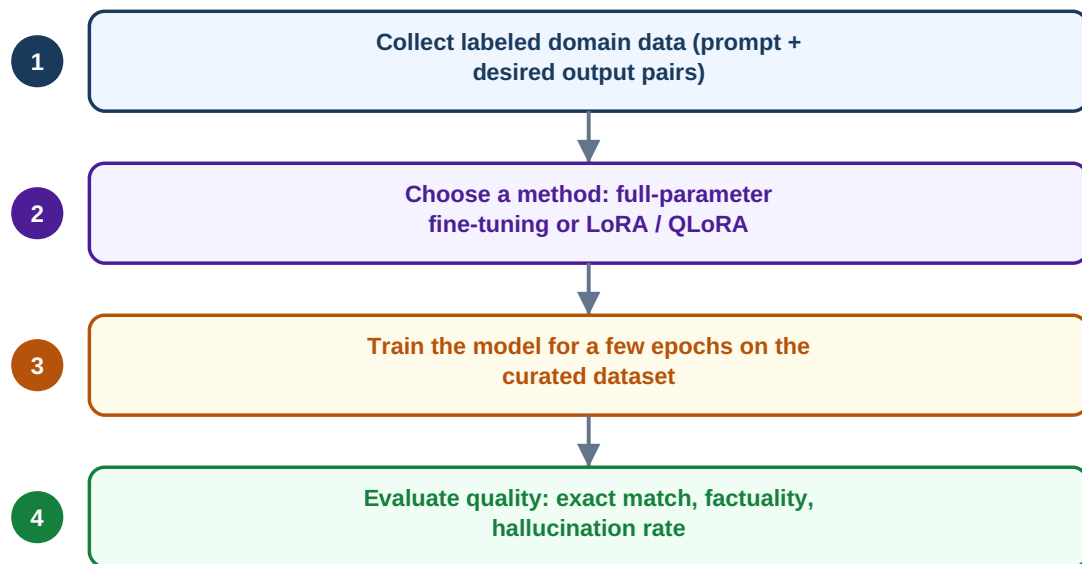


Figure 12.1 — The four-step supervised fine-tuning process.

How Fine-Tuning Addresses the Three Problems

- **Private data:** once the LLM is trained on private data, that data becomes part of its parametric knowledge, so the model can now answer related questions.
- **Recent data:** partially solved — every time new information arrives, you must fine-tune again with the new data, which is repetitive and costly.
- **Hallucination:** can be reduced by adding tricky prompts to the training data along with "I don't know" as the desired output, explicitly training the model to stick to known facts.

COMMON MISTAKE

Assuming fine-tuning is a one-time fix. In fast-changing domains, fine-tuning has to be repeated every time new information is added or old information becomes outdated — making it an ongoing cost, not a one-off investment.

Major Problems With Fine-Tuning

1. **Computationally expensive:** even on a small dataset, you are still training a very large model, which costs money and compute.
2. **Requires strong technical expertise:** proper AI engineers and data scientists are needed to run the process correctly.
3. **Poor fit for fast-changing domains:** every catalog update, new document, or removed item requires re-running fine-tuning and paying for it again.

🔄 Quick Revision — Fine-Tuning

- Fine-tuning re-trains a pre-trained LLM on a smaller, domain-specific dataset.
- Supervised Fine-Tuning uses labeled prompt/desired-output pairs; Continued Pre-Training is unsupervised.
- RLHF and LoRA/QLoRA are other common fine-tuning techniques.
- Supervised fine-tuning follows 4 steps: collect data, choose a method, train, evaluate.
- Fine-tuning can address all three problems (private data, recent data, hallucination) at some level.
- Major downsides: computationally expensive, requires specialized expertise, and is impractical for frequently-changing information.

3. In-Context Learning: A Cheaper Alternative

Definition

In-context learning is a core capability of large language models (like GPT-3, Claude, LLaMA) where the model learns to solve a task purely by seeing examples inside the prompt, without updating its weights at all.

How It Works: Few-Shot Prompting

Instead of relying only on parametric knowledge, you show the model a handful of example input/output pairs directly inside the prompt. For instance, to teach sentiment analysis, you might provide three labeled examples ("I love this phone" → Positive, "This app crashes a lot" → Negative, "The camera is amazing" → Positive) and then ask the model to classify a new sentence ("I hate the battery life"). The model learns the pattern from the examples and applies it to the new input. This technique of providing examples inside the prompt is called *few-shot prompting*.

REAL WORLD INSIGHT

In-context learning is an *emergent property* — a behavior that suddenly appears in a system once it reaches a certain scale and complexity, even though it was never explicitly programmed or expected from the individual components. Small models like GPT-1 and GPT-2 did not reliably show this behavior. It was only with GPT-3 (around 175 billion parameters) that researchers observed the model could reliably learn from examples given purely in the prompt. This was documented in the landmark paper "*Language Models are Few-Shot Learners*."

The paper's key argument was that, unlike fine-tuning — which needs a labeled dataset of 10,000 to 1,000,000+ rows — humans can generally perform a new language task from just a few examples and simple instructions. GPT-3 demonstrated that a sufficiently large language model could do the same. Later models (GPT-3.5, GPT-4) became even better at in-context learning thanks to further alignment techniques such as supervised fine-tuning and RLHF applied on top.

IMPORTANT

In-context learning is emergent, which means it is not guaranteed to give good results on every kind of task. It works well for many tasks but is not a universal guarantee of quality.

Quick Revision — In-Context Learning

- In-context learning lets a model solve a task by seeing examples in the prompt, without updating its weights.
- Few-shot prompting is the technique of including example input/output pairs inside the prompt.
- In-context learning is an emergent property that appeared reliably only at GPT-3's scale (~175B parameters).
- The landmark paper on this topic is 'Language Models are Few-Shot Learners.'
- Being an emergent property means the technique is not guaranteed to perform well on every task.

4. What Is RAG?

Definition

RAG (Retrieval Augmented Generation) is a way to make a language model smarter by giving it extra information at the time you ask your question. Instead of providing task-solving examples the way few-shot prompting does, RAG sends the entire relevant context needed to answer a specific question, injected directly into the prompt.

Why It Exists

RAG is an evolution of in-context learning: rather than teaching the model how to solve a task through examples, you hand it exactly the knowledge it needs for a single question. For example, if a student has a doubt about Gradient Descent inside a two-hour Linear Regression lecture, rather than sending the entire two-hour transcript, you send just the relevant transcript segment (say, minutes 5 to 25) as context alongside the student's question. The model then combines this injected context with its own parametric knowledge to produce the response.

Prompt Structure

```
# System instruction
You are a helpful assistant. Answer the question only from the
provided context. If the context is insufficient, just say you don't know.

# Context (retrieved from the knowledge base)
{retrieved_context}

# User's question
{user_query}
```

Grounding the model's response strictly against the supplied context (and instructing it to say "I don't know" when the context is insufficient) is what keeps RAG-based answers accurate and reduces hallucination.

INTERVIEW TIP

If asked to define RAG in one line: "RAG combines Information Retrieval (finding the right pieces of external knowledge) with Text Generation (an LLM producing a response), so the model answers using both its parametric knowledge and freshly retrieved, non-parametric context."

🔄 Quick Revision — What Is RAG

- RAG makes an LLM smarter by injecting extra, relevant information into the prompt at question time.
- Unlike few-shot prompting (examples of how to solve a task), RAG provides the actual knowledge needed to answer one specific question.
- A RAG prompt typically contains a system instruction, the retrieved context, and the user's query.
- Instructing the model to say 'I don't know' when context is insufficient reduces hallucination.
- RAG = combination of Information Retrieval + Text Generation.

5. The Four Stages of a RAG Pipeline

At a high level, RAG is built from two foundational computer science concepts: **Information Retrieval** (a long-standing field concerned with finding relevant information) and **Text Generation** (popularized by LLMs). Broadly speaking, any RAG-based system can be broken down into four stages:

1. Indexing

2. Retrieval

3. Augmentation

4. Generation

INDEXING

The process of creating an external knowledge base. This is where all the context you will eventually inject into a prompt ultimately comes from.

RETRIEVAL

The process of understanding a user's query, searching the external knowledge base, and pulling out the chunks that are most relevant to answering that query.

AUGMENTATION

The process of combining the user's query with the retrieved context to construct the final prompt sent to the LLM — you are augmenting the LLM's parametric knowledge with extra, retrieved knowledge.

GENERATION

The final stage, where the LLM reads the augmented prompt and uses its text-generation capability along with in-context learning to produce the response.

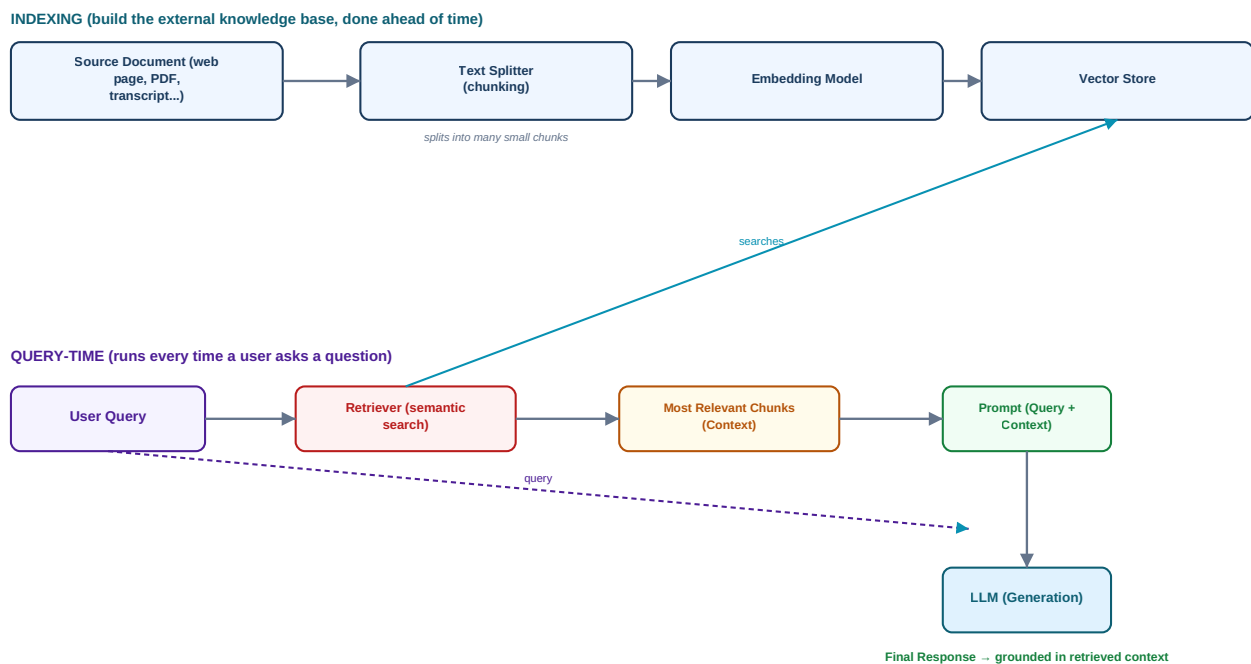


Figure 12.1 — The complete RAG pipeline: indexing builds the knowledge base ahead of time; retrieval, augmentation, and generation run on every user query.

IMPORTANT

Indexing happens once (or whenever the knowledge base needs updating). Retrieval, Augmentation, and Generation happen on every single user query. This is the key structural distinction to remember about the RAG pipeline.

🔄 Quick Revision — The Four RAG Stages

- RAG combines Information Retrieval (an older CS field) with Text Generation (popularized by LLMs).
- The four stages are: Indexing, Retrieval, Augmentation, Generation.
- Indexing builds the external knowledge base ahead of time.
- Retrieval finds the most relevant chunks for a specific query.
- Augmentation combines the query and retrieved context into a final prompt.
- Generation is the LLM producing the final response using the augmented prompt.

6. Indexing: Building the External Knowledge Base

Definition

Indexing is the process of preparing your knowledge base so that it can be efficiently searched at query time. It is made up of four sub-steps.

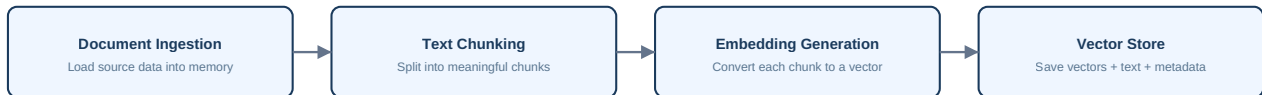


Figure 12.2 — The four sub-steps of indexing.

STEP 1: DOCUMENT INGESTION

Loading the source knowledge into memory. Wherever the underlying data lives — a server, Google Drive, AWS S3 — it must first be fetched and loaded. LangChain provides many built-in Document Loaders for this (PyPDF Loader, YouTube Loader, WebBase Loader, and others). After this step, the document exists in memory as a string.

STEP 2: TEXT CHUNKING

Breaking the large document into smaller, semantically meaningful chunks, for two main reasons:

- There is a limit on how many tokens an LLM can process in a single prompt, so a very large document cannot be sent as-is.
- Semantic search quality degrades when performed against very large documents.

Chunks should not be split abruptly — ideally each chunk stays focused on a single topic. LangChain offers several Text Splitters for this: the Recursive Character Text Splitter (the most common), Semantic Chunker, and format-specific splitters for HTML or Markdown.

STEP 3: EMBEDDING GENERATION

Converting each chunk into a dense vector using an embedding model (e.g. OpenAI Embeddings, Sentence Transformers). This captures the semantic meaning of each chunk numerically, which is what makes later semantic search possible — semantic search always happens between vectors, not raw text.

STEP 4: VECTOR STORAGE

Storing each vector alongside its original chunk text and any metadata inside a vector database. Common local options include FAISS and Chroma; cloud-based options include Pinecone, Weaviate, Milvus, and Qdrant. Once stored, this vector store becomes the external knowledge base used for all future retrieval.

BEST PRACTICE

Aim for chunks that are semantically coherent (ideally focused on one topic each) rather than mechanically splitting by a fixed character count with no regard for meaning — this directly affects retrieval quality later.

🔄 Quick Revision — Indexing

- Indexing has 4 sub-steps: Document Ingestion, Text Chunking, Embedding Generation, Vector Storage.
- Document Ingestion loads source data into memory as a string, using LangChain Document Loaders.
- Text Chunking exists because of LLM token limits and because semantic search works poorly on huge documents.
- Embedding Generation converts each chunk into a dense vector capturing its semantic meaning.
- Vector Storage saves vectors, original text, and metadata in a vector database (e.g. FAISS, Chroma, Pinecone).
- The completed vector store is the external knowledge base RAG retrieves from at query time.

7. Retrieval: Finding the Right Context

Definition

Retrieval is the real-time process of finding the most relevant pieces of information from a pre-built index (the vector store) based on the user's question — effectively asking "of all the knowledge I have, which 3-to-5 chunks are most helpful for answering this specific query?"

How the Retriever Works (4 Steps)

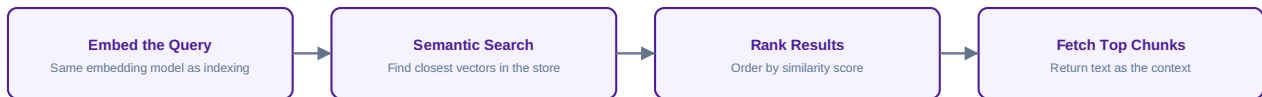


Figure 12.3 — The four internal steps performed by a Retriever component.

STEP 1: EMBED THE QUERY

The user's query is converted into an embedding vector using the exact same embedding model that was used to embed the chunks during indexing, so both live in the same vector space.

STEP 2: SEMANTIC SEARCH

The retriever searches the vector store for vectors closest to the query embedding. This can be a simple semantic search, or a more advanced technique such as MMR (Maximal Marginal Relevance) or contextual compression.

STEP 3: RANK RESULTS

The retrieved candidates are ranked by closeness — from a simple similarity metric like cosine similarity to more advanced re-ranking algorithms.

STEP 4: FETCH TOP CHUNKS

The text of the top-ranked chunks is fetched and becomes the context that will be injected into the prompt.

INTERVIEW TIP

Be ready to explain why the same embedding model must be used for both indexing and query-time embedding: if the models differ, the resulting vectors won't live in a comparable space, and similarity search will return meaningless results.

Quick Revision — Retrieval

- Retrieval is the real-time process of finding the most relevant chunks for a specific query.
- The retriever's 4 steps: embed the query, semantic search, rank results, fetch top chunks.
- The same embedding model used during indexing must be used to embed the query.
- Advanced retrieval techniques include MMR and contextual compression.
- Ranking can use simple cosine similarity or more advanced re-ranking algorithms.

8. Augmentation and Generation

Augmentation

Once the retriever has produced the most relevant chunks (the context), the query is taken as-is and combined with that context to construct the final prompt. This is the augmentation stage — you are literally adding extra, non-parametric knowledge on top of the LLM's parametric knowledge so it can answer properly.

```
# Final augmented prompt sent to the LLM
You are a helpful assistant. Answer the questions only from the
provided context. If the context is insufficient, just say you don't know.

# Context
{retrieved_context}

# Question
{user_query}
```

Generation

The augmented prompt is sent to the LLM. The model reads the prompt, uses its text-generation capability together with in-context learning, and produces the final response — grounded in both its parametric knowledge and the freshly retrieved context.

IMPORTANT

This four-stage pipeline (Indexing → Retrieval → Augmentation → Generation) is a basic RAG setup. It can be improved with more advanced techniques (better retrieval strategies, re-ranking, query rewriting, and more), which will be covered in future chapters.

Quick Revision — Augmentation and Generation

- Augmentation combines the user's query with the retrieved context into one final prompt.
- The augmented prompt explicitly instructs the model to answer only from the provided context.
- Generation is the LLM producing the final response from the augmented prompt.
- The LLM uses both parametric knowledge and the retrieved (non-parametric) context to answer.
- This basic four-stage pipeline can later be extended with more advanced RAG techniques.

9. How RAG Solves the Three Original Problems

Problem	How RAG Solves It
Private Data	The external knowledge base is built directly from your own data, so retrieved context — and therefore the model's answers — naturally comes from your private data.
Recent / Current Data	Simply add new documents (recent articles, news, updated course content) into the external knowledge base. No retraining is required — you only need to embed the new document and add it to the vector store.
Hallucination	The model is explicitly instructed to answer only from the provided context and to say "I don't know" when the context is insufficient, grounding its response and greatly reducing the chance of fabricated answers.

RAG vs. Fine-Tuning

Aspect	Fine-Tuning	RAG
Model training required	Yes — the model itself is retrained	No — the model's weights are never touched
Data requirement	Labeled dataset (1,000 to 1,000,000+ prompt/output pairs)	Raw documents, used as-is
Cost of updates	High — must retrain every time data changes	Low — just re-embed and add the new document
Complexity	High — needs AI engineers / data scientists	Lower — no training pipeline needed

REAL WORLD INSIGHT

RAG is generally the cheaper and simpler alternative to fine-tuning for keeping an LLM up to date with private or fast-changing information, because it avoids any actual model training. Fine-tuning still has its place — particularly when you need the model to change its underlying behavior or reasoning style, not just its access to facts.

Quick Revision — RAG vs. the Three Problems

- Private data problem: solved because the knowledge base is built directly from your own documents.
- Recent data problem: solved by adding new documents to the vector store — no retraining needed.
- Hallucination problem: reduced by grounding responses strictly in the retrieved context.
- RAG needs no model training, unlike fine-tuning.
- RAG is generally cheaper and simpler to maintain than fine-tuning, especially for frequently changing data.

K. Key Takeaways

- **Parametric knowledge** is all an LLM knows, stored as weights; prompting is the only way to access it, but it fails on private data, recent data, and hallucinates.
- **Fine-tuning** retrains the LLM on domain-specific data, solving all three problems at some level, but is computationally expensive, expertise-heavy, and costly to repeat for frequent updates.
- **In-context learning** lets a model learn from examples inside the prompt without weight updates; it is an emergent property that appeared reliably at GPT-3 scale.
- **RAG** evolves in-context learning by injecting the actual retrieved context needed to answer a query, rather than task-solving examples.
- RAG has **four stages**: Indexing (build the knowledge base), Retrieval (find relevant chunks), Augmentation (build the final prompt), Generation (LLM produces the answer).
- **Indexing** has four sub-steps: document ingestion, text chunking, embedding generation, vector storage.
- The **Retriever** embeds the query, performs semantic search, ranks results, and fetches the top chunks as context.
- RAG solves all three original problems — private data, recent data, hallucination — without any model training, making it cheaper and simpler than fine-tuning.

R. Revision Sheet

Parametric Knowledge

All knowledge an LLM has, stored as numbers in its weights and biases after pre-training.

Three Problems

Prompting alone fails on private data, recent/current data, and can hallucinate confidently incorrect answers.

Fine-Tuning

Re-training a pre-trained LLM on a smaller domain-specific dataset; expensive, expertise-heavy, costly to repeat.

In-Context Learning

A model solves a task by seeing examples in the prompt, without weight updates; an emergent property at GPT-3 scale.

RAG

Injects the actual retrieved context needed for a query into the prompt, combining Information Retrieval with Text Generation.

Indexing

Ingest → Chunk → Embed → Store: builds the external knowledge base once, ahead of time.

Retrieval

Embed query → Search → Rank → Fetch: finds the most relevant chunks for one specific query.

Augmentation & Generation

Combine query + context into a final prompt; the LLM generates a response grounded in that context.

RAG vs. Fine-Tuning

RAG needs no retraining, is cheaper to update, and is simpler — but fine-tuning better changes underlying model behavior.

I. Interview Questions

Q1. What is parametric knowledge, and why can't an LLM answer questions about data outside its training set?

Q2. Explain the three scenarios where prompting an LLM directly fails to produce a good response.

Q3. What is fine-tuning, and what are the key differences between Supervised Fine-Tuning and Continued Pre-Training?

Q4. What are the major drawbacks of fine-tuning as a solution for keeping an LLM's knowledge current?

Q5. Define in-context learning. Why is it described as an "emergent property"?

Q6. How is RAG different from few-shot prompting, even though both are forms of in-context learning?

Q7. Walk through the four stages of a RAG pipeline and explain what happens in each.

Q8. Why is a large document split into chunks before embedding, instead of embedding it as a whole?

Q9. Why must the same embedding model be used both during indexing and at query time?

Q10. How does RAG reduce hallucination compared to a model relying purely on its parametric knowledge?

F. Further Reading

- Brown et al., "*Language Models are Few-Shot Learners*" (the GPT-3 paper introducing in-context learning)
- LangChain Documentation — Document Loaders, Text Splitters, Vector Stores, and Retrievers
- LangChain Documentation — Retrieval Augmented Generation (RAG) tutorials
- Overview articles on Parameter-Efficient Fine-Tuning: LoRA and QLoRA
- Vector database documentation: FAISS, Chroma, Pinecone